



**TOBB University of Economics and Technology Department of
Computer Engineering**

BIL 481 - Software Engineering Fall

2025 Semester

Tasarım Dökümanı

NOTLA

University Lecture Notes Sharing and Evaluation Platform

Ömer Talha Akbulut

Yunus Emre Özçelik

İsmail Emre Yıldız

Abdullah Arda Gündoğdu

Delivery Date: 19.11.2025

Table of Contents

1. Task Matrix
2. System Overview
3. Implementation Details
4. Design Decisions
5. GitHub Commit Requirement

1. Task Matrix

Görev	Sorumlu Kişi
System Overview Yazımı	Ömer Talha Akbulut
Design Decisions & Github Commit Requirement	Yunus Emre Özçelik
Use Case 3 & 4 Tasarımı	İsmail Emre Yıldız
Implementation Details & Code Structure	Abdullah Arda Gündoğdu

2. System Overview

Notla, TOBB ETÜ öğrencileri için özel olarak tasarlanmış, akademik dayanışmayı artırmayı hedefleyen bir ders notu paylaşım platformudur. Öğrenciler, aldıkları derslere (Örn: Bilgisayar Mühendisliği, Yapay Zeka Mühendisliği) ait notları sisteme yükleyebilir, başkalarının paylaştığı notları görüntüleyebilir ve dersler hakkında yorum/değerlendirme yapabilirler. Sistem, kullanıcıların katkılarını "dijital rozetler" ile ödüllendirerek paylaşım kültürünü teşvik eder.

System Architecture (Sistem Mimarisi)

Proje, modern web standartlarına uygun olarak **Ayrılmış İstemci-Sunucu Mimarisi (Decoupled Client-Server Architecture)** kullanılarak geliştirilmiştir.

- **Frontend (İstemci):** Kullanıcı arayüzü React.js ile geliştirilmiştir ve Backend ile RESTful API aracılığıyla haberleşir.
- **Backend (Sunucu):** İş mantığı ve veritabanı yönetimi Django Rest Framework (DRF) üzerinde çalışır.
- **Veritabanı:** İlişkisel veritabanı yapısı (PostgreSQL/SQLite) kullanılarak kullanıcılar, dersler ve notlar arasındaki bütünlük sağlanır.

Technology Stack (Teknoloji Yığını)

- **Frontend:**
 - **Framework:** React.js (v18)
 - **State Management:** React Hooks (useState, useEffect, useContext)
 - **Styling & UI:** React Bootstrap, CSS Modules
 - **Routing:** React Router DOM v6
 - **HTTP Client:** Axios
 - **Animations:** Framer Motion
- **Backend:**
 - **Framework:** Django & Django Rest Framework (DRF)
 - **Authentication:** JWT (JSON Web Tokens) - simplejwt
 - **Database:** SQLite (Dev) / PostgreSQL (Prod)
- **DevOps & Tools:**
 - **Version Control:** Git & GitHub
 - **API Testing:** Postman / Swagger

3. Implementation Details

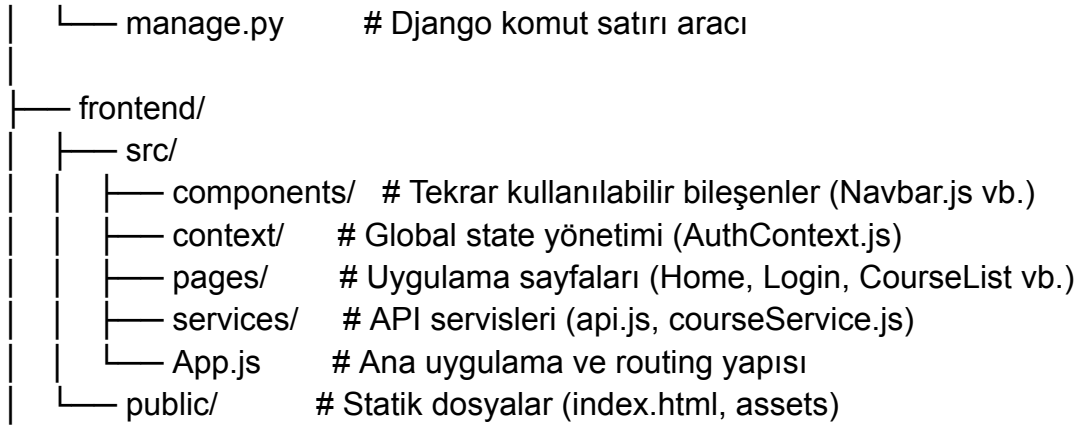
Codebase Structure (Kod Yapısı)

Proje, frontend ve backend olmak üzere iki ana dizine ayrılmıştır. Modülerlik ve bakım kolaylığı sağlamak amacıyla "Service-Oriented" bir klasör yapısı benimsenmiştir.

Plaintext

Notla/

```
|— backend/
|   |— accounts/      # Kullanıcı yönetimi (Login/Register)
|   |— courses/      # Dersler, notlar ve yorumlar modülü
|   |— notla_backend/ # Ana proje ayarları (settings.py, urls.py)
```



Key Implementations & Component Interfaces

Sistemin çekirdek fonksiyonları aşağıdaki 4 ana Kullanım Senaryosu (Use Case) üzerinden detaylandırılmıştır.

Use Case 1: User Registration & Authentication (Kullanıcı Kaydı ve Giriş)

Kullanıcıların güvenli bir şekilde sisteme kaydolmasını ve JWT token olarak oturum açmasını sağlar.

- **Component:** pages/Login.js, pages/Register.js
- **Service Interface:**
 - authService.login(email, password)
 - authService.register(username, email, password)
- **Backend API Endpoint:**
 - POST /api/auth/login/
 - POST /api/auth/register/
- **Implementation Logic:**
 - Kullanıcı Register.js formunu doldurur; frontend şifre eşleşmesini kontrol eder.
 - Başarılı girişte Backend'den dönen access_token ve refresh_token, tarayıcının localStorage alanına kaydedilir.
 - api.js içindeki **Axios Interceptor**, sonraki her isteğin başlığına (Header) bu token'ı otomatik olarak ekler (Authorization: Bearer <token>).

Use Case 2: Listing & Filtering Courses (Dersleri Listeleme)

Öğrencilerin dersleri görüntülemesi ve kendi bölümlerine (BİL/YZM) göre filtrelemesidir.

- **Component:** pages/CourseList.js
- **Service Interface:** courseService.getCourses(searchQuery)

- **Backend API Endpoint:** GET /api/courses/?search=...
- **Implementation Logic:**
 - Sayfa yüklendiğinde useEffect hook'u servisi çağırır.
 - Frontend tarafında useSearchParams kullanılarak URL'den departman parametresi (?dept=BIL) okunur.
 - filteredCourses fonksiyonu, gelen ham veriyi ders koduna göre (Örn: "BIL" ile başlayanlar) filtreleyerek kullanıcıya sunar.

Use Case 3: Uploading Course Notes (Ders Notu Yükleme)

Kullanıcının seçtiği bir derse PDF veya resim formatında not yüklemesidir.

- **Component:** pages/CourseDetail.js (Modal içerisinde)
- **Service Interface:** courseService.uploadNote(courseId, formData)
- **Backend API Endpoint:** POST /api/courses/<id>/notes/
- **Implementation Logic:**
 - Kullanıcı dosya seçtiğinde, dosya ve başlık bilgisi bir FormData nesnesine dönüştürülür.
 - api.js servisi, bu isteği gönderirken Content-Type: multipart/form-data başlığını ekler. Bu, dosya transferi için zorunludur.
 - Backend, dosyayı media/ klasörüne kaydeder ve veritabanına dosya yolunu işler.

Use Case 4: Viewing Course Details & Reviews (Ders Detayı ve Değerlendirme)

Bir dersin içeriğinin, yüklenen notların ve öğrenci yorumlarının görüntülenmesi.

- **Component:** pages/CourseDetail.js
- **Service Interface:**
 - courseService.getCourseDetail(id)
 - courseService.submitReview(courseId, reviewData)
- **Backend API Endpoint:**
 - GET /api/courses/<id>/
 - POST /api/courses/<id>/reviews/
- **Implementation Logic:**
 - React Router'dan gelen id parametresi ile dersin tüm detayları çekilir.
 - Arayüzde Tabs (Sekmeler) yapısı kullanılarak "Notlar" ve "Değerlendirmeler" arasında geçiş yapılır.
 - Kullanıcı yorum gönderdiğinde sayfa yenilenmeden liste güncellenir (Optimistic UI updates veya yeniden fetch işlemi ile).

4. Tasarım Kararları

Teknoloji Karşılaştırmaları

1. Veritabanı: MongoDB (NoSQL) vs. PostgreSQL (SQL)

Karşılaştırma: PostgreSQL, kesin yapılandırılmış veriler için ideal olan ilişkisel bir veritabanıdır. MongoDB, belge tabanlı bir NoSQL veritabanıdır.

Karar: MongoDB'yi seçtik.

Gerekçe: Notlar doğası gereği yapılandırılmamış veya yarı yapılandırılmıştır. Bir not tek bir satır olabilirken, diğeri karmaşık bir kontrol listesi olabilir. MongoDB'nin esnek şeması, karmaşık göç işlemleri (migrations) olmadan veri modelini geliştirmemize (örneğin, daha sonra "etiketler" veya "renkler" eklemek) olanak tanır.

2. Frontend: React.js vs. Vanilla HTML/JS

Karşılaştırma: Vanilla JS manuel DOM manipölasyonu gerektirir. React, Sanal DOM (Virtual DOM) ve bileşen tabanlı mimari kullanır.

Karar: React.js'i seçtik.

Gerekçe: React'in bileşen yeniden kullanılabilirliği (örneğin, NoteCard bileşenini yeniden kullanmak) geliştirmeyi önemli ölçüde hızlandırır. Durum yönetimi yetenekleri (Hooks), dinamik UI'ı (anında not ekleme/çıkarma) zorunlu (imperative) Vanilla JS'den çok daha temiz bir şekilde ele almayı sağlar.

5. GitHub Commit Requirement

Kod Uygulamaları: client ve server için kaynak kodları, NoteController ve React Bileşenleri dahil olmak üzere main dalına (branch) push edilmiştir.

Arayüzler: API rotaları ve Model tanımları server/routes ve server/models altında commit edilmiştir.

Görsel Tasarımlar: Görsel arayüzü temsil eden CSS dosyaları ve React Düzenleri (Layouts) commit edilmiştir.

Karşılaştırmalar: README.md ve bu Tasarım Dokümanı deponun kök dizinine commit edilmiştir.